第6讲作业三和作业四：软件可信性度量

刘小凡 10244602411

一、作业三：计算可靠性属性的可信度量值

1. 题目与数据

四个软件系统均有可靠性属性 y，每个系统包含五个子属性 x_1, x_2, x_3, x_4, x_5。两种模型使用相同的子属性权重：

子属性	x_1	x_2	x_3	x_4	x_5
权重 γ_i（或 ω_i）	0.15	0.20	0.20	0.25	0.20

课件给出的两种模型为：

$$\text{模型 1：}\quad y_1 = \prod_{i=1}^{5} x_i^{\gamma_i} = x_1^{0.15} x_2^{0.20} x_3^{0.20} x_4^{0.25} x_5^{0.20}。$$

$$\text{模型 2：}\quad y_2 = \left(\sum_{i=1}^{5} \omega_i x_i^{-\rho_y} \right)^{-1/\rho_y},\qquad \rho_y > 0。$$

模型 1 不使用参数 ρ_y；模型 2 按照每行给出的 ρ_y 计算。以系统 1 的第一行为例：

$$y_1 = 8.6^{0.15} \times 9.1^{0.20} \times 9.2^{0.20} \times 8.8^{0.25} \times 8.9^{0.20} = 8.927694。$$

$$y_2 = \left(0.15 \times 8.6^{-0.01} + 0.20 \times 9.1^{-0.01} + 0.20 \times 9.2^{-0.01} + 0.25 \times 8.8^{-0.01} + 0.20 \times 8.9^{-0.01}\right)^{-100} = 8.927671。$$

2. 计算结果

系统	行	x_1	x_2	x_3	x_4	x_5	ρ_y	模型 1：y_1	模型 2：y_2
1	1	8.6	9.1	9.2	8.8	8.9	0.01	8.927694	8.927671
1	2	8.6	9.1	9.2	8.8	9.9	0.55	9.119863	9.114381
2	1	6.8	7.9	5.9	6.6	6.1	0.01	6.614913	6.614570
2	2	6.8	7.9	5.9	6.6	6.1	0.55	6.614913	6.596355
3	1	9.1	9.9	8.9	8.8	7.8	0.01	8.859329	8.859071
3	2	9.1	9.9	8.9	8.8	7.8	0.55	8.859329	8.845068
4	1	3.5	4.2	5.6	4.9	5.2	0.01	4.695127	4.694559
4	2	3.5	4.2	5.6	4.9	5.2	0.55	4.695127	4.663374

数据说明：课件表中系统 1 的第一行 $x_5 = 8.9$，第二行 $x_5 = 9.9$。因此，尽管模型 1 不使用 ρ_y，这两行因输入数据不同，y_1 仍然不同。上表逐行使用课件原值。

二、作业四：计算权重向量并选优

1. 题目与判断矩阵

C919 飞行控制软件的五个可信属性按顺序为：实时性、可靠性、可生存性、可维护性、功能性。课件给出的正互反判断矩阵为：

$$A=\begin{bmatrix} 1 & 1/2 & 3 & 2 & 1/2 \\ 2 & 1 & 2 & 3 & 2 \\ 1/3 & 1/2 & 1 & 2 & 1/3 \\ 1/2 & 1/3 & 1/2 & 1 & 2 \\ 2 & 1/2 & 3 & 1/2 & 1 \end{bmatrix}。$$

分别使用右特征向量法 EV、对数最小二乘法 LLSM、卡方最小二乘法 CSM 求权重向量，再用强度指标 TD 选择结果。

2. 计算方法

（1）右特征向量法 EV。 求最大特征值 $\lambda_{\max}$ 对应的右特征向量 v，满足 $Av=\lambda_{\max}v$；再将 v 的各项归一化，使权重之和为 1。程序使用幂迭代求解。本题 $\lambda_{\max}\approx 5.723387$。

（2）对数最小二乘法 LLSM。 求判断矩阵每一行元素的几何平均数，并将所得数值归一化：

$$g_i=\left(\prod_{j=1}^{5}a_{ij}\right)^{1/5},\qquad w_i=\frac{g_i}{\sum_{k=1}^{5}g_k}。$$

（3）CSM。 按照课件第 49—51 页给出的迭代程序，从 $w_i=1/5$ 开始，计算：

$$d_i=\sum_{j=1}^{5}\left[(1+a_{ji}^2)\frac{w_i}{w_j}-(1+a_{ij}^2)\frac{w_j}{w_i}\right]。$$

选取 $|d_i|$ 最大的下标 m，计算下面的 U、D，将 w_m 乘以 $\sqrt{U/D}$，再将向量归一化：

$$U=\sum_{j\neq m}(1+a_{mj}^2)\frac{w_j}{w_m},\qquad D=\sum_{j\neq m}(1+a_{jm}^2)\frac{w_m}{w_j}。$$

重复上述过程，直到 $\max_i|d_i|\le 10^{-10}$。

（4）强度指标。 按课件，分别对三种方法计算：

$$TD=\sum_{i=1}^{5}\sum_{j=1}^{5}\left|a_{ij}-\frac{w_i}{w_j}\right|。$$

TD 越小，表示权重比值与判断矩阵元素之间的总偏差越小。程序使用未截断的权重计算 TD，表中结果保留六位小数。

3. 计算结果与选择

权重顺序为：实时性、可靠性、可生存性、可维护性、功能性。

方法	w_1	w_2	w_3	w_4	w_5	TD
EV	0.197468	0.319251	0.124711	0.146451	0.212119	12.375243
LLSM	0.200815	0.349640	0.119325	0.129404	0.200815	**11.408461**
CSM（课件迭代程序）	0.205334	0.350042	0.112428	0.131433	0.200764	11.521132

由于 $TD_{\mathrm{LLSM}}<TD_{\mathrm{CSM}}<TD_{\mathrm{EV}}$，按课件的强度指标选择 **LLSM**。因此最优权重向量为：

$$W^{\mathrm{LLSM}}=(0.200815,\ 0.349640,\ 0.119325,\ 0.129404,\ 0.200815)。$$

课件方法说明：课件第 48 页写出的 CSM 平方目标函数，与第 49—51 页给出的迭代程序并不一致。本作业遵照课件中的迭代程序，表内 CSM 数值是该程序的结果。对第 48 页的目标函数 $\sum_{i,j}(a_{ij}/(w_i/w_j)-1)^2$，这个结果的函数值约为 10.022852；例如取归一化后的 $(0.196581, 0.349791, 0.123346, 0.142440, 0.187842)$，函数值约为 9.431575。因此，不能把课件迭代结果断言为第 48 页所写目标函数的最小解。

三、C++ 程序

程序内置课件数据，完成两道作业的计算。运行结束后按回车退出，方便截图。使用 g++ 编译和运行：

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic -static 第6讲作业三四.cpp -o 第6讲作业三四.exe
.\第6讲作业三四.exe
```

```cpp
#include <algorithm>
#include <array>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <stdexcept>

using namespace std;

constexpr size_t N = 5;
using Vector = array<double, N>;
using Matrix = array<Vector, N>;

// 作业三的模型 1：子属性加权乘积。
double model1(const Vector& x, const Vector& weight) {
    double result = 1.0;
    for (size_t i = 0; i < N; ++i) {
        result *= pow(x[i], weight[i]);
    }
    return result;
}

// 作业三的模型 2：按照子属性权重及 rho 合成。
double model2(const Vector& x, const Vector& weight, double rho) {
    double sum = 0.0;
    for (size_t i = 0; i < N; ++i) {
        sum += weight[i] * pow(x[i], -rho);
    }
    return pow(sum, -1.0 / rho);
}

Vector normalize(const Vector& v) {
    const double sum = accumulate(v.begin(), v.end(), 0.0);
    Vector result{};
    for (size_t i = 0; i < N; ++i) result[i] = v[i] / sum;
    return result;
}

Vector multiply(const Matrix& a, const Vector& v) {
    Vector result{};
    for (size_t i = 0; i < N; ++i) {
        for (size_t j = 0; j < N; ++j) result[i] += a[i][j] * v[j];
    }
    return result;
}

// EV：幂迭代求最大特征值对应的右特征向量，再归一化。
Vector eigenvectorMethod(const Matrix& a) {
    Vector w{};
    w.fill(1.0 / N);
    for (int iteration = 0; iteration < 100000; ++iteration) {
        const Vector next = normalize(multiply(a, w));
        double difference = 0.0;
        for (size_t i = 0; i < N; ++i)
            difference = max(difference, abs(next[i] - w[i]));
        w = next;
        if (difference < 1e-13) return w;
    }
    throw runtime_error("EV iteration did not converge");
}

// LLSM：每行元素的几何平均数归一化。
Vector logarithmicLeastSquares(const Matrix& a) {
```

```
    Vector roots{};
    for (size_t i = 0; i < N; ++i) {
        double product = 1.0;
        for (size_t j = 0; j < N; ++j) product *= a[i][j];
        roots[i] = pow(product, 1.0 / N);
    }
    return normalize(roots);
}

// CSM：对应课件第 49–51 页的迭代过程。
Vector chiSquareMethod(const Matrix& a) {
    const double epsilon = 1e-10;
    Vector w{};
    w.fill(1.0 / N);
    for (int iteration = 0; iteration < 100000; ++iteration) {
        size_t selected = 0;
        double largest = -1.0;
        for (size_t i = 0; i < N; ++i) {
            double value = 0.0;
            for (size_t j = 0; j < N; ++j) {
                value += (1.0 + a[j][i] * a[j][i]) * (w[i] / w[j])
                       - (1.0 + a[i][j] * a[i][j]) * (w[j] / w[i]);
            }
            if (abs(value) > largest) {
                largest = abs(value);
                selected = i;
            }
        }
        if (largest <= epsilon) return w;

        double up = 0.0, bottom = 0.0;
        for (size_t j = 0; j < N; ++j) {
            if (j == selected) continue;
            up += (1.0 + a[selected][j] * a[selected][j]) * (w[j] / w[selected]);
            bottom += (1.0 + a[j][selected] * a[j][selected]) * (w[selected] / w[j]);
        }
        w[selected] *= sqrt(up / bottom);
        w = normalize(w);
    }
    throw runtime_error("CSM iteration did not converge");
}

// 课件中的强度指标 TD：值越小，权重向量越优。
double totalDeviation(const Matrix& a, const Vector& w) {
    double td = 0.0;
    for (size_t i = 0; i < N; ++i)
        for (size_t j = 0; j < N; ++j)
            td += abs(a[i][j] - w[i] / w[j]);
    return td;
}

void printVector(const Vector& v) {
    cout << '(';
    for (size_t i = 0; i < N; ++i) {
        if (i != 0) cout << ", ";
        cout << v[i];
    }
    cout << ')';
}

int main() {
    const Vector attributeWeight = {0.15, 0.20, 0.20, 0.25, 0.20};
    struct Case {
        int system, row;
        Vector x;
```

```
        double rho;
    };
    const array<Case, 8> cases = {{
        {1, 1, {{8.6, 9.1, 9.2, 8.8, 8.9}}, 0.01},
        {1, 2, {{8.6, 9.1, 9.2, 8.8, 9.9}}, 0.55},
        {2, 1, {{6.8, 7.9, 5.9, 6.6, 6.1}}, 0.01},
        {2, 2, {{6.8, 7.9, 5.9, 6.6, 6.1}}, 0.55},
        {3, 1, {{9.1, 9.9, 8.9, 8.8, 7.8}}, 0.01},
        {3, 2, {{9.1, 9.9, 8.9, 8.8, 7.8}}, 0.55},
        {4, 1, {{3.5, 4.2, 5.6, 4.9, 5.2}}, 0.01},
        {4, 2, {{3.5, 4.2, 5.6, 4.9, 5.2}}, 0.55}
    }};
    const Matrix a = {{
        {{1.0,     1.0/2, 3.0, 2.0, 1.0/2}},
        {{2.0,     1.0,   2.0, 3.0, 2.0}},
        {{1.0/3,   1.0/2, 1.0, 2.0, 1.0/3}},
        {{1.0/2,   1.0/3, 1.0/2, 1.0, 2.0}},
        {{2.0,     1.0/2, 3.0, 1.0/2, 1.0}}
    }};

    cout << fixed << setprecision(6);
    cout << "Homework 3: attribute measurement\n";
    cout << "System  Row  rho       Model 1       Model 2\n";
    for (const auto& c : cases) {
        cout << setw(6) << c.system << setw(5) << c.row
             << setw(7) << setprecision(2) << c.rho << setprecision(6)
             << setw(14) << model1(c.x, attributeWeight)
             << setw(14) << model2(c.x, attributeWeight, c.rho) << '\n';
    }

    const Vector ev = eigenvectorMethod(a);
    const Vector llsm = logarithmicLeastSquares(a);
    const Vector csm = chiSquareMethod(a);
    const double tdEV = totalDeviation(a, ev);
    const double tdLLSM = totalDeviation(a, llsm);
    const double tdCSM = totalDeviation(a, csm);
    const Vector av = multiply(a, ev);
    const double lambda = accumulate(av.begin(), av.end(), 0.0);
    cout << "\nHomework 4: weight vectors and TD\n";
    cout << "EV   = "; printVector(ev); cout << "  TD = " << tdEV << '\n';
    cout << "LLSM = "; printVector(llsm); cout << "  TD = " << tdLLSM << '\n';
    cout << "CSM  = "; printVector(csm); cout << "  TD = " << tdCSM << '\n';
    cout << "EV largest eigenvalue = " << lambda << '\n';
    if (tdEV <= tdLLSM && tdEV <= tdCSM) cout << "Best by TD: EV\n";
    else if (tdLLSM <= tdCSM) cout << "Best by TD: LLSM\n";
    else cout << "Best by TD: CSM\n";

    cout << "\nPress Enter to exit..." << flush;
    cin.get();
    return 0;
}
```

实际运行结果

```
Homework 3: attribute measurement
System  Row  rho       Model 1       Model 2
     1    1   0.01      8.927694      8.927671
     1    2   0.55      9.119863      9.114381
     2    1   0.01      6.614913      6.614570
     2    2   0.55      6.614913      6.596355
     3    1   0.01      8.859329      8.859071
     3    2   0.55      8.859329      8.845068
     4    1   0.01      4.695127      4.694559
     4    2   0.55      4.695127      4.663374

Homework 4: weight vectors and TD
EV   = (0.197468, 0.319251, 0.124711, 0.146451, 0.212119)  TD = 12.375243
LLSM = (0.200815, 0.349640, 0.119325, 0.129404, 0.200815)  TD = 11.408461
CSM  = (0.205334, 0.350042, 0.112428, 0.131433, 0.200764)  TD = 11.521132
EV largest eigenvalue = 5.723387
Best by TD: LLSM

Press Enter to exit...
```

四、运行截图

